asknget Backend

Developer Guide for Frontend Engineers and Stakeholders

Version 0.5 | 2026-09-19 | Django 6.1 + django-ninja 1.7 | API base: `/api/v1/`

This guide explains how the asknget backend is put together, which API to call at each step of the user journeys, the technologies involved, and how to run it locally with **uv**. It is written for the frontend team first, but the overview and flow sections are intended to be readable by product and QA stakeholders too.

Note. The project runs on an agile cadence and the Figma design changes frequently. This document reflects the design snapshot of 2026-09-11 and the code on branch `main`. Live, always-current API documentation is served by the backend itself at `/api/v1/docs` (Swagger UI). When this PDF and Swagger disagree, Swagger wins.

Contents

1. What the backend serves 4

Delivery status . . . 4

2. Technology stack 5

3. Architecture 5

Repository layout . . . 5

The three-file pattern inside every app . . . 5

Request lifecycle . . . 6

Design principles the frontend can rely on . . . 6

4. Running the backend locally with uv 7

Prerequisites . . . 7

First-time setup . . . 7

Environment variables (backend/.env) . . . 7

Useful URLs once running . . . 7

Everyday commands . . . 8

Demo accounts (after seed_demo; password DemoPass123!) . . . 8

5. Authentication and error handling 9

Tokens . . . 9

Session lifecycle the frontend must implement . . . 9

Status codes and error shape . . . 9

6. API flows by user journey 10

6.1 Customer sign-up and verification . . . 10

6.2 Login . . . 10

6.3 My Account - Personal Information . . . 11

6.4 My Account - Password Manager and Logout . . . 11

6.5 My Account - Manage Addresses . . . 11

6.6 My Account - Payment Methods . . . 11

6.7 Seller sign-up and KYC application . . . 13

6.8 Admin: KYC review (no screens designed - Swagger or a simple admin UI) . . . 14

6.9 Storefront: browse and product page . . . 15

6.10 Cart and checkout . . . 15

6.11 My Orders and tracking . . . 17

6.12 Search and recommendations . . . 17

6.13 Seller dashboard . . . 18

6.14 Support and marketing . . . 19

7. Endpoint reference 20

Seller portal (Bearer token of a seller account) . . . 21

Catalogue (public; send a Bearer token to get is_wishlisted) . . . 22

Cart, checkout, tracking, support (guests: X-Cart-Token header) . . . 23
Admin (Bearer token of a staff account) . . . 24
Key schemas . . . 24
8. Data conventions for the frontend **25**
9. Working with the API day to day **25**
Trying calls in Swagger . . . 25
Generating a typed client . . . 25
Where to look when something is unclear . . . 26
Roadmap (subject to sprint planning) . . . 26

1. What the backend serves

asknget is a **multi-vendor e-commerce marketplace**. The backend exposes one JSON API that serves three audiences:

- **Customers** - browse the catalogue, manage cart, checkout, track orders, maintain profile, addresses, saved cards and wishlist.
- **Sellers (vendors)** - sign up, complete a 5-step KYC application (personal, business, representative, banking, review), upload documents, respond to reviewer requests, and activate a store once approved.
- **Admins** - review KYC applications, request more information, approve or reject, and manage catalogue data. Admin screens are not in the design yet; the Django admin site covers this for now.

Delivery status

Area	Status	Notes
Data model (38 tables) + migrations	Implemented	Derived from all 38 Figma screens; see docs/data-model.md
Django admin for every model	Implemented	http://127.0.0.1:8000/admin/
Auth: register, login, JWT, refresh, logout	Implemented	/api/v1/auth/*
OTP verification (email, SMS), password change/reset	Implemented	SMS provider is a console stub until an SMS gateway is chosen
Profile, avatar, addresses, payment methods	Implemented	/api/v1/me/*
Seller application (5 steps, autosave), documents, submit/resubmit, status timeline, store activation	Implemented	/api/v1/seller/*, /api/v1/lookups/*
Admin KYC review: queue, request-info, approve/reject, document review	Implemented	/api/v1/admin/seller-applications/* (staff JWT)
Catalogue: categories, products + filters, PDP, related, reviews, wishlist	Implemented	/api/v1/categories, /products/*, /me/wishlist
Cart (members + guests), vouchers, delivery methods, checkout	Implemented	/api/v1/cart/*, /checkout - order is created pending_payment
My Orders, cancel, re-order, public tracking, admin order status	Implemented	/api/v1/me/orders/*, /orders/track, /admin/orders/*
Banners, newsletter, contact form, site info	Implemented	/api/v1/banners, /newsletter/subscribe, /contact, /site-info
AI search: hybrid keyword + semantic ranking, typo tolerance, suggestions, query log	Implemented	/api/v1/products?q=, /search/suggest - pgvector; offline embedder by default, Voyage/OpenAI by env
Recommendations: similar, frequently bought together, trending, 'Recommended for you'	Implemented	/api/v1/products/{slug}/similar|frequently-bought-together, /recommendations
Docker packaging: API image (uv), Postgres+pgvector, embeddings worker	Implemented	compose.yaml (+ compose.override.yaml for live reload)
Payment provider: card tokenisation, capture, webhooks, 'save card'	Planned	Next sprint. Checkout records the chosen method only; orders stay unpaid until then.
Seller dashboard: home KPIs, products wizard + review, inventory, orders (per-store fulfilment), customers	Implemented	/api/v1/seller/dashboard|products|inventory|orders|customers|search; /admin/products (Figma 2026-09-18)
Seller portal: Returns, Chat, Analytics page, Notifications, Team/Managers, social login, invoice PDF	Not designed	Sidebar entries only - see data-model.md section 10 Q20-Q34
Vendor sub-roles / permissions, premium tier, vendor-customer chat	Not designed	'Team/Managers' and 'Chat' now appear in the seller sidebar; still no screens

2. Technology stack

Layer	Technology	Why / notes
Language	Python 3.13	Pinned in backend/.python-version
Package / env manager	uv 0.12	Replaces pip + venv; lockfile uv.lock guarantees identical installs
Web framework	Django 6.1	ORM, migrations, admin, auth primitives
API framework	django-ninja 1.7	Fast, typed REST on top of Django; pydantic v2 validation; auto OpenAPI/Swagger
Validation	pydantic 2	Every request body is a typed schema; errors are returned as 422 with field paths
Auth	PyJWT (HS256)	Short-lived access tokens; opaque rotating refresh tokens stored hashed
Database	PostgreSQL 17 + pgvector (target) / SQLite (dev fallback)	Set DATABASE_URL to switch; psycopg 3 driver with connection pool; pgvector for embeddings, full-text + trigram for keywords
Embeddings	hash (offline, default) / Voyage AI / OpenAI	EMBEDDING_PROVIDER; one vector per product (discovery.ProductEmbedding); swap provider then embed_products --all
Containers	Docker Compose: db, web (gunicorn), worker	Two-stage image from astral/uv; dev override bind-mounts the code for live reload
Field encryption	cryptography (Fernet)	ID/passport numbers and bank account numbers encrypted at rest
Images	Pillow	Avatar, product, banner, store images
Config	python-dotenv + dj-database-url	All settings via environment variables / backend/.env
Docs	Swagger UI (built in)	/api/v1/docs and /api/v1/openapi.json
Tests	Django TestCase	End-to-end through the real URL config; 121 tests, green on SQLite and on Postgres+pgvector inside the container

3. Architecture

Repository layout

```
asknget/                    git root  (github.com/AunSyedShah/asknget)
+-- docs/
|   +-- data-model.md       every table, field, enum, relationship, open questions
|   +-- figma/snapshot.py   dumps the Figma screens as text; --check detects design drift
|   +-- build_developer_guide.py   generates this PDF
+-- backend/
    +-- pyproject.toml, uv.lock, .env.example, README.md
    +-- asknget/            Django project root  (manage.py lives here)
        +-- asknget/        settings.py, urls.py, api.py (NinjaAPI root, error handlers)
        +-- core/           shared: base models, EncryptedTextField, validators, JWT auth, seed commanc
        +-- accounts/       User, OTP, refresh tokens, terms audit, addresses, payment methods
        +-- catalog/        categories, brands, products, variants/options, reviews, wishlist
        +-- sellers/        seller application (KYC), documents, info requests, stores
        +-- orders/         cart, vouchers, delivery methods, orders, status timeline, payments
        +-- marketing/      banners, newsletter, contact messages
```

The three-file pattern inside every app

Each Django app is split the same way so that a change lands in exactly one place. This is the main reason the codebase tolerates frequent design changes:

File	Owns	Changes when...
schemas.py	Request/response shapes and field-level validation (pydantic)	a form on the design changes
services.py	Business rules: uniqueness, cooldowns, state machines, side effects, transactions	a flow or rule changes
api.py	Routers - thin functions: parse, call a service, return	an endpoint is added or renamed
models.py	Database tables and constraints	the data model changes (needs a migration)
tests.py	End-to-end tests through the real URLs	any of the above changes

Request lifecycle

```
Frontend  --HTTP JSON-->  Django URLconf (/api/v1/)  -->  NinjaAPI
                                                          |
                              1. Auth class (JWTAuth) validates Bearer token -> request.auth = User
                              2. pydantic schema validates + normalises the body (422 on failure)
                              3. Router function calls a service
                              4. Service enforces rules, touches the ORM inside a transaction
                              5. Response schema serialises the result (explicit field list only)
                                                          |
Frontend  <--JSON + status code (200/201/204 or 4xx {"detail": ...})--
```

Design principles the frontend can rely on

- **Explicit responses.** Every response schema lists its fields; nothing leaks by accident (no password hashes, no internal tokens).
- **Separate In / PatchIn / Out schemas.** Create payloads require their fields; PATCH payloads are fully optional and *omitted is not null* - only keys you send are updated.
- **Enums are shared.** Dropdown values (address type, gender, order status, application status...) come from one source and appear in Swagger.
- **Rules live server-side.** The frontend should validate for UX, but the API is the authority (cooldowns, uniqueness, default switching, state transitions).
- **Versioned base path.** Everything is under `/api/v1/`. A breaking change means a new version, not an edit.

4. Running the backend locally with uv

Prerequisites

- Git, and **uv** (https://docs.astral.sh/uv/). uv installs the pinned Python 3.13 for you; no system Python required.
- Optional: PostgreSQL 15+ if you want parity with production. Without it the project falls back to SQLite automatically.

First-time setup

```
git clone https://github.com/AunSyedShah/asknget.git
cd asknget/backend

uv sync                                  # creates .venv and installs exactly what uv.lock says
cp .env.example .env                     # then edit .env (see below)

cd asknget                               # the Django project root (contains manage.py)
uv run python manage.py migrate          # create the database schema
uv run python manage.py seed_lookups     # categories, brands, banks, delivery methods
uv run python manage.py seed_demo        # demo accounts, store, 13 products, vouchers (dev only)
uv run python manage.py createsuperuser  # optional: your own admin login
uv run python manage.py runserver        # http://127.0.0.1:8000
```

Environment variables (backend/.env)

Variable	Required	Purpose
SECRET_KEY	prod	Django secret; also signs access tokens. A dev default exists but never use it beyond localhost.
DEBUG	no	1 (default) or 0
ALLOWED_HOSTS	prod	Comma-separated hostnames
DATABASE_URL	no	postgres://user:pass@host:5432/asknget - omit to use SQLite (backend/asknget/db.sqlite3)
FIELD_ENCRYPTION_KEY	yes*	Fernet key for encrypted PII. *Required before any seller banking/ID data is saved. Generate: python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
JWT_ACCESS_TTL_MINUTES	no	Default 15
JWT_REFRESH_TTL_DAYS	no	Default 30
OTP_TTL_MINUTES / OTP_RESEND_COOLDOWN_SECONDS / OTP_MAX_ATTEMPTS	no	Defaults 10 / 45 / 5

Useful URLs once running

URL	What
http://127.0.0.1:8000/api/v1/docs	Swagger UI - try every endpoint in the browser (use Authorize with a Bearer token)
http://127.0.0.1:8000/api/v1/openapi.json	OpenAPI 3 spec - feed it to a client generator (openapi-typescript, orval, etc.)
http://127.0.0.1:8000/api/v1/health	Liveness probe
http://127.0.0.1:8000/admin/	Django admin - inspect and edit any table

Everyday commands

```
uv run python manage.py test                # run all tests
uv run python manage.py test accounts       # one app
uv run python manage.py makemigrations      # after changing models
uv run python manage.py migrate
uv add <package>                            # add a dependency (updates pyproject + uv.lock)
python ../docs/figma/snapshot.py --check    # has the Figma design changed since we last looked?
```

Demo accounts (after seed_demo; password DemoPass123!)

Email	Role	Use it for
customer@asknget.demo	Customer	Browse, cart, checkout, My Orders, wishlist (has a saved address)
seller@asknget.demo	Active seller	Seller portal with an approved, active store ('Huda Beauty') and 13 products
admin@asknget.demo	Staff / superuser	Django admin and every /api/v1/admin/* endpoint (KYC review, order status)

Vouchers seeded: `SAVE20` (20% over $50) and `WELCOME5` ($5, once per user).

Note. OTP emails are printed to the runserver console in development (console email backend); SMS codes are logged. Look at the terminal running `runserver` to find the 6-digit code while testing.

5. Authentication and error handling

Tokens

Token	Lifetime	Where to keep it	How it is used
access_token (JWT)	15 min	Memory (not localStorage if you can avoid it)	Header on every protected call: Authorization: Bearer <access_token>
refresh_token (opaque)	30 days	Secure httpOnly cookie or secure storage	POST /auth/refresh to get a new pair. It is single-use: every refresh returns a new refresh token and the old one is dead.

Session lifecycle the frontend must implement

#	Screen / user action	API call	Backend behaviour
1	App start	POST /auth/refresh {refresh_token}	Rotates the token; 401 means the user must log in again
2	Any protected call returns 401 'Access token expired'	POST /auth/refresh, then retry the original call once	Standard silent-refresh interceptor
3	User presses Logout (popup)	POST /auth/logout {refresh_token} or {all_devices: true}	Revokes the refresh token(s). Discard both tokens client-side.
4	Password changed or reset	-	All refresh tokens are revoked server-side; other devices are signed out on their next refresh

Status codes and error shape

Code	Meaning	Body
200 / 201 / 204	OK / created / no content	Resource, or empty for 204
400	Business rule failed on user input (wrong OTP, wrong current password)	{"detail": "..."}
401	Missing/invalid/expired token, or bad credentials	{"detail": "..."}
403	Account suspended / not allowed	{"detail": "..."}
404	Resource not found or not yours	{"detail": "Not found"}
409	Conflict - email or phone already registered	{"detail": "..."}
422	Validation failed	{"detail": "summary", "errors": [{"field": "phone", "message": "..."}]}
429	Rate limited - OTP resend cooldown or too many attempts	{"detail": "Please wait 32s ..."}

Example 422 body:

```
{
  "detail": "phone: Phone must be in international format, e.g. +923001234567;
             confirm_password: Passwords do not match",
  "errors": [
    {"field": "phone", "message": "Phone must be in international format, e.g. +923001234567"},
    {"field": "confirm_password", "message": "Passwords do not match"}
  ]
}
```

Use `errors[].field` to highlight inputs and `detail` as a fallback toast. A field of `null` means the error concerns the whole payload.

6. API flows by user journey

Each table reads top to bottom: what the user does on the screen, which API the frontend calls, and what the backend does. Every endpoint below is live in Swagger except the payment-provider steps, which are marked *next sprint*.

6.1 Customer sign-up and verification

#	Screen / user action	API call	Backend behaviour
1	Sign-up form submitted	POST /auth/register {email, password, confirm_password, phone?, full_name?}	Creates the user, returns user + tokens (201). Email lower-cased, phone normalised to E.164. 409 if taken.
2	'Verify Your Email' screen shown	POST /auth/otp/send {channel: 'email', purpose: 'signup_email'} (Bearer)	Sends 6-digit code; response has target_masked, expires_at, resend_available_at (45 s cooldown)
3	User types the 6 digits	POST /auth/otp/verify {channel, purpose, target: email, code}	Marks email_verified_at. 400 wrong code, 429 after 5 wrong attempts.
4	'Resend code in 00:45' countdown ends	POST /auth/otp/send again	429 with remaining seconds if called too early
5	Mobile OTP screen (same pattern)	channel: 'sms', purpose: 'signup_phone'	Sets phone_verified_at

Note. Registration already returns tokens, so the user is logged in immediately; there is no need to call /auth/login after sign-up. The `user` object in the response is the same shape as `GET /me`, with profile fields empty until the user fills them in on the Personal Information page.

6.2 Login

#	Screen / user action	API call	Backend behaviour
1	Sign-in form	POST /auth/login {email, password}	200 with user + tokens; 401 bad credentials; 403 suspended
2	'Forgot password'	POST /auth/password/forgot {email}	Always 200 (does not reveal whether the email exists); emails a code if it does
3	Reset form	POST /auth/password/reset {email, code, new_password, confirm_password}	Sets password, revokes all sessions. User must sign in again.

6.3 My Account - Personal Information

#	Screen / user action	API call	Backend behaviour
1	Page opens	GET /me	Full profile incl. email_verified, phone_verified, member_since (Account Information panel)
2	'Save Changes' on Personal Details	PATCH /me {full_name?, gender?, date_of_birth?, profession?, country?, city?, address_line?}	Partial update - only sent keys change. Enums and past-date checks return 422.
3	'Change Photo'	POST /me/avatar (multipart, field name 'file')	PNG/JPG/GIF, max 2 MB; returns updated profile with avatar_url
4	Change email or phone	OTP flow (6.1) - not PATCH /me	Email/phone are identity fields and require re-verification (endpoint planned)

6.4 My Account - Password Manager and Logout

#	Screen / user action	API call	Backend behaviour
1	'Update Password'	POST /me/password {current_password, new_password, confirm_password}	400 if current is wrong; on success all other sessions are signed out
2	Logout popup confirmed	POST /auth/logout {refresh_token}	204; clear tokens client-side

6.5 My Account - Manage Addresses

#	Screen / user action	API call	Backend behaviour
1	Tab opens	GET /me/addresses	List, default first
2	'Save Address' in popup	POST /me/addresses {type, label?, full_name, phone, street_address, unit_floor?, city, province?, postal_code, country?, delivery_instructions?, is_default?}	201. The first address always becomes default; is_default=true moves the default.
3	'Edit' on a card	PATCH /me/addresses/{id}	Partial update
4	Set as default	POST /me/addresses/{id}/default	Exactly one default is enforced by a DB constraint
5	Delete	DELETE /me/addresses/{id}	204; soft-deleted; if it was default, the newest remaining address becomes default

6.6 My Account - Payment Methods

Note. Card numbers and CVV never reach this API. The frontend tokenises the card with the payment provider's SDK and sends the token plus display data. The provider is not chosen yet; the contract below is provider-agnostic.

#	Screen / user action	API call	Backend behaviour
1	Tab opens	GET /me/payment-methods	List with display_name like 'VISA ....2389'

#	Screen / user action	API call	Backend behaviour
2	'Add Card' in popup	POST /me/payment-methods {type: 'card', provider, provider_token, card_brand, card_holder_name, last4, exp_month, exp_year, is_default?}	201. For type=card all card fields are required and expiry must be in the future. PayPal/Apple/Google Pay need only type, provider, provider_token.
3	Set default / delete	POST .../{id}/default, DELETE .../{id}	Same semantics as addresses

6.7 Seller sign-up and KYC application

#	Screen / user action	API call	Backend behaviour
1	'Create your seller account' (account type radio + form + terms checkbox)	POST /auth/seller/register {email, phone, password, confirm_password, account_type, accept_terms: true}	Creates user (is_seller) + draft application + terms audit rows; returns tokens. 422 if accept_terms is false.
2	Email and mobile OTP screens	Same as 6.1	
3	Wizard opens / resumes	GET /seller/application	Everything: current_step, per-step complete/missing_fields/missing_documents, all saved values (ID and account numbers masked), documents, can_submit
4	Dropdowns (business type, role, account type, bank)	GET /lookups/seller, GET /lookups/banks?country=PK, GET /categories	Public; option lists live in one place on the backend
5	Step 1-4 'Save & Continue' (also on blur / autosave)	PATCH /seller/application/steps/basic-info | business-info | representative | banking	Partial: send only the fields on screen; nothing is required until submit. Returns the full application so the stepper can re-render. 409 once submitted.
6	'+ Upload Document' (ID, company registration, bank letter, proof of address)	POST /seller/application/documents (multipart: type, file)	PDF/JPG/PNG up to 10 MB. Re-uploading the same type creates version n+1; old version kept for reviewers.
7	'+ Add Another Director'	POST /seller/application/directors, PATCH|DELETE .../directors/{id}	Up to 10
8	Step 5 'Submit'	POST /seller/application/submit {confirm_accuracy: true, accept_terms: true}	422 lists exactly what is missing per step; otherwise draft -> submitted, application number ASK-SLR-xxxxxx issued, returns the status payload
9	Application Submitted / Status page	GET /seller/application/status	application_number, status, status_headline, 5-stage timeline (done | current | pending | skipped), events, open_info_request
10	'We Need More Information' banner + 'Required Updates' list	GET /seller/application (open_info_request.items[])	Each item is a document to replace or a field group to edit; `resolved` flips as the seller acts
11	'Update & Resubmit'	POST .../documents (same type), PATCH the step, then POST /seller/application/resubmit {confirm_accuracy: true}	422 until every item is resolved; then more_info_required -> resubmitted and the application locks again
12	'Activate Your Seller Account' - Store URL check	GET /seller/store/slug-available?slug=huda-beauty	Returns available + a free suggestion
13	'Activate & Start Selling'	POST /seller/store/activate {display_name?, slug?, accept_seller_agreement: true, confirm_info_accurate: true}	Only when approved; store becomes active at asknget.com/store/<slug>; application -> active

Note. Per account type: **individual** sellers skip the Representative step and the company-registration document; sole proprietors and registered companies complete all four steps and four documents. This lives in sellers/rules.py and is a one-line change.

6.8 Admin: KYC review (no screens designed - Swagger or a simple admin UI)

#	Screen / user action	API call	Backend behaviour
1	Review queue	GET /admin/seller-applications?status=submitted&q;=&page;=	Staff JWT. Drafts are excluded; oldest submission first
2	Open an application	GET /admin/seller-applications/{id}	Full detail incl. decrypted ID / account numbers, every document version, info requests, event log
3	Start review	POST .../{id}/start-review	submitted -> under_review; records reviewer
4	Ask for more information	POST .../{id}/request-info {message, items: [{kind: document|field_group, target, instruction}]}	-> more_info_required; flags documents replacement_requested; seller sees the banner
5	Accept / reject a single document	POST .../{id}/documents/{docId}/review {status, note}	Does not change application status
6	Approve	POST .../{id}/approve {note?}	-> approved; creates the Store (pending_activation) with a unique slug
7	Reject	POST .../{id}/reject {reason}	-> rejected (terminal); reason shown to the seller

6.9 Storefront: browse and product page

#	Screen / user action	API call	Backend behaviour
1	Landing page	GET /banners?placement=hero|top_discount, GET /products/featured|best-sellers|new-arrivals|top-discounted, GET /categories	Public. Cards carry is_wishlisted when a Bearer token is sent
2	Categories page: sidebar	GET /products/filters	Category tree with counts, brands, colours, price range - one call
3	Categories page: grid	GET /products?category=beauty&brand;=nars&color;=Black&price;_min=&price;_max=∈_stock=true&q;=&sort;=new|price_asc|price_desc|rating|popular&page;=1&per;_page=12	{items, total, page, per_page, pages}; category includes sub-categories; repeat brand/color params for multi-select
4	Product page	GET /products/{slug}	Options (Size/Shade/Color) with values, variants (option_value_ids, price, stock), key_benefits, usage_steps, ratings breakdown, store
5	Pick options -> variant	client-side	Find the variant whose option_value_ids match the selected values; disable combos with in_stock=false
6	Reviews tab	GET /products/{slug}/reviews?sort=new|top|low&page;=, GET /products/{slug}/reviews/summary	Approved reviews only; names abbreviated ('Ayesha K.')
7	'Submit Review'	POST /products/{slug}/reviews (multipart: name?, email?, rating, body, files[])	Guests must send name + email; members default to their profile. Stored pending; 'Verified Purchase' when the member has a delivered order for it
8	'You may also like'	GET /products/{slug}/similar (or /related)	similar = embedding neighbours, topped up by same category/brand; related = the old rule-only version
9	'Frequently bought together'	GET /products/{slug}/frequently-bought-together	Order co-occurrence (cancelled orders ignored), cached 10 min; falls back to similar
10	Wishlist heart / Wishlist tab	PUT|DELETE /me/wishlist/{productId}, GET /me/wishlist	PUT is idempotent

6.10 Cart and checkout

Note. Guests: the first call to any /cart endpoint returns `cart_token`. Store it (localStorage) and send it as the `X-Cart-Token` header on every cart/checkout call. When the user signs in, send both the Bearer token and the header once - the guest cart merges into their account cart and the token is retired.

#	Screen / user action	API call	Backend behaviour
1	Cart page opens	GET /cart	items (with variant_summary, image, stock), order_note, voucher, delivery_method, summary {subtotal, shipping, discount, total}, can_checkout, problems[]
2	'Add to Cart' / 'Buy Now'	POST /cart/items {variant_id, qty}	Merges quantity if already present; clamps to stock; 409 when sold out
3	Quantity stepper / remove	PATCH /cart/items/{id} {qty}, DELETE /cart/items/{id}	409 if qty exceeds stock
4	'Order Notes', 'Delivery Type'	PATCH /cart {order_note?, delivery_method_id?}	Shipping cost in the summary follows the chosen method (default: Standard)
5	'Discount Voucher -> Apply'	POST /cart/voucher {code}, DELETE /cart/voucher	404 unknown; 422 with the reason (expired, minimum not met, already used)
6	Checkout page: delivery options	GET /delivery-methods	'Standard Delivery 5 - 7 working days $10.99', 'Express ...'
7	'Place Order'	POST /checkout {contact, address | address_id, ship_to_different_address?, shipping_address?, delivery_method_id, payment_method_type, payment_method_id?, special_note?}	201 {order, detail}. Re-checks stock under a lock, snapshots everything, reserves stock, redeems the voucher, clears the cart, emails confirmation. Order is pending_payment / unpaid.
8	Card entry / 'Save card for future checkout'	*next sprint*	The PSP SDK will tokenise the card client-side; the API will never see PAN/CVV
9	'Order Placed!' screen	use the checkout response	detail carries the 'Thank You, John Ali...' copy; order.order_number for the confirmation

6.11 My Orders and tracking

#	Screen / user action	API call	Backend behaviour
1	Tabs All / Ongoing / Completed / Cancelled	GET /me/orders?tab=all|ongoing|completed|cancelled&page;=	Each card: order_number, status_label, group, items (qty, variant_summary, prices), can_cancel, can_reorder, items[].can_review
2	'View Detail' / 'Track Order' (member)	GET /me/orders/{id}	Adds addresses, delivery method, totals, events[] timeline (status, label, note, location, time)
3	'Cancel'	POST /me/orders/{id}/cancel {reason?}	Allowed until the order is processing; restores stock; 409 afterwards
4	'Re-Order'	POST /me/orders/{id}/reorder	Puts still-available lines back in the cart and returns the cart; unavailable ones are listed in problems[]
5	'Leave Review'	POST /products/{slug}/reviews (6.9)	items[].can_review is true on delivered, un-reviewed lines
6	Track Order page (guest)	POST /orders/track {order_number, email}	Public; matches the email used at checkout; returns status + events
7	Ops move an order (no screen)	POST /admin/orders/{id}/status {status, note?, location?}	confirmed -> processing -> shipped -> out_for_delivery -> delivered; each writes a timeline event

6.12 Search and recommendations

Search is **hybrid**: every sidebar filter is applied first (so results stay exact), then the remaining candidates are ranked by two independent rankers fused with Reciprocal Rank Fusion - keyword (Postgres websearch full-text over a per-product text card + trigram similarity on the name for typos) and semantic (cosine nearest neighbours of the query embedding via pgvector). A product found by both rises; an exact brand/SKU still wins on keywords alone; nonsense queries return nothing because each ranker has a relevance floor. Recommendations reuse the same embeddings: a shopper's **taste profile** is the weighted mean of what they viewed (1), wishlisted (3), added to cart (4) or bought (6), halved every 30 days; 'Recommended for you' is its nearest neighbours minus everything they already interacted with. Behaviour is recorded server-side (discovery.ProductEvent) from the existing endpoints - the frontend sends no tracking calls; guests are keyed by X-Cart-Token.

#	Screen / user action	API call	Backend behaviour
1	Header search box (typeahead)	GET /search/suggest?q=lip	Up to 5 products, brands and categories matching the prefix (trigram-tolerant on Postgres)
2	'Search Products' / results page	GET /products?q=...&category;=&price;_max=&sort;=	sort defaults to relevance when q is given; any explicit sort (price_asc...) is honoured over the matched set. Logged with result_count (0 = nothing found)
3	PDP 'You may also like'	GET /products/{slug}/similar?limit=6	Semantic neighbours across the whole public catalogue

#	Screen / user action	API call	Backend behaviour
4	PDP / cart 'Frequently bought together'	GET /products/{slug}/frequently-bought-together?limit=4	From real orders; similar() until enough orders exist
5	Home 'Recommended for you'	GET /recommendations?limit=8 (+ Bearer or X-Cart-Token)	strategy=personalised with seeds[] ('Because you viewed ...') or strategy=trending until there is history
6	Ops: switch to a real embedding model	EMBEDDING_PROVIDER=voyage + VOYAGE_API_KEY, then manage.py embed_products --all	Default 'hash' provider is offline/lexical (dev + tests); the worker service keeps embeddings fresh as products change

6.13 Seller dashboard

Every route below needs a seller **with an active store** (403 `no_active_store` otherwise, so the portal can send the user back to the application status page). Everything shown is computed **per store**: a buyer order that spans several stores appears to each seller as only *their* lines (a **store order**) with its own fulfilment status; the buyer-facing order status is derived from the parts. KPI cards return `{value, previous, change_pct}` for the requested `period` (today | week | month | quarter | year). Full field contract: docs/sprint-3-plan.md.

#	Screen / user action	API call	Backend behaviour
1	Seller Dashboard (home)	GET /seller/dashboard?period=month	sales (booked) vs revenue (delivered), orders, products, low stock, 12-month series, performance (views, conversion, AOV, customer growth), latest 5 orders, stock breakdown
2	Header 'Search anything...'	GET /seller/search?q=	Own products, order numbers/customers, customers
3	Products table + tabs + KPI cards	GET /seller/products?status=&q;=&brand;=&category;=&stock;=&sort;=, GET /seller/products/kpis	Tabs all/active/pending/draft/rejected/archived; sold_qty and earned_cents per row
4	'+ Add New Product' step 1 Basic Info	POST /seller/products {name, category_id, brand_id?, tags[], short_description}	Creates a draft with one default variant; category_id is the leaf (subcategory)
5	Steps 2-4 (Media, Pricing & Inventory, Shipping)	POST .../images (multipart), PUT .../images/order, PATCH /seller/products/{id} (price, compare-at, threshold, shipping), PUT .../options {options}, PATCH .../variants/{vid} {sku, price_cents, stock_qty}	Options regenerate variants for every combination, keeping matching SKUs/stock; stock changes are logged movements
6	Step 5 Review -> submit	POST /seller/products/{id}/status {action: submit}	422 lists what is missing (image, price, variant, stock); product becomes pending until staff approve
7	Staff review (no screen yet)	GET /admin/products?status=pending, POST /admin/products/{id}/approve | reject {reason}	Approve publishes; reject stores the reason the seller sees under the Rejected tab
8	Row actions	unpublish/archive/unarchive via .../status, POST .../duplicate, DELETE (draft/rejected/archived only), GET .../analytics	Analytics: views, sold, earned, conversion, 30-day series

#	Screen / user action	API call	Backend behaviour
9	Inventory	GET /seller/inventory?tab=all|low|out|recent, GET /seller/inventory/kpis, POST /seller/inventory/{variant_id}/adjust {set | delta, note}, GET .../movements, GET /seller/inventory/export.csv	on_hand = sum of variant stock; reserved = paid but unshipped units; every adjustment is a StockMovement
10	Orders Overview	GET /seller/orders/overview?period=	KPIs, monthly delivered/pending/refunded, status share, latest 5
11	Orders table	GET /seller/orders?tab=pending|processing|shipped|delivered|cancelled|returned&q;=&date;_from=&date;_to=	Unpaid orders are not shown to sellers (Q27)
12	View Order	GET /seller/orders/{id}	Customer info, addresses, payment, per-store timeline (Placed, Payment Confirmed, Processing, Packed, Shipped, Delivered), this store's items, summary with multi_store flag, allowed_next
13	'Update Status' / 'Cancel Order'	POST /seller/orders/{id}/status {status, note?}, POST .../cancel {reason}	pending -> processing -> packed -> shipped -> delivered; returned after delivery; cancel/return restore stock
14	'Print / Download Invoice', 'Export'	GET /seller/orders/{id}/invoice, GET /seller/orders/export.csv	Invoice data only - PDF layout is a UI concern until designed (Q28)
15	Customers Overview / table / detail	GET /seller/customers/overview, GET /seller/customers?tab=new|returning|one_time&q;=&status;=, GET /seller/customers/{key}	Derived from orders; key = user id or guest email; type/status rules in sprint-3-plan.md section 5
16	'Add Note', 'Export Customers'	POST /seller/customers/{key}/notes {body}, DELETE .../notes/{id}, GET /seller/customers/export.csv	Notes are private to the store

6.14 Support and marketing

#	Screen / user action	API call	Backend behaviour
1	Contact Us 'Send a Message' / 'Contact Seller Support'	POST /contact {first_name, last_name, email, phone?, subject, message}	Subjects from GET /site-info; linked to the seller application when subject=seller_support and signed in
2	Footer 'Subscribe Now'	POST /newsletter/subscribe {email}	201 new, 200 already subscribed
3	Footer contact info / 'We Accept'	GET /site-info	Values maintained in Django admin (Site settings)

7. Endpoint reference

All paths are prefixed with `/api/v1`. 'Auth' = requires `Authorization: Bearer`; 'store' = seller with an active store; 'staff' = is_staff.

Method + path	Auth	Request body	Success	Errors
POST /auth/register	-	RegisterIn	201 AuthOut	409, 422
POST /auth/seller/register	-	SellerRegisterIn	201 AuthOut	409, 422
POST /auth/login	-	LoginIn	200 AuthOut	401, 403
POST /auth/refresh	-	RefreshIn	200 TokenOut	401
POST /auth/logout	yes	LogoutIn	204	401
POST /auth/otp/send	optional	OtpSendIn	200 OtpSendOut	422, 429
POST /auth/otp/verify	-	OtpVerifyIn	200 MessageOut	400, 429
POST /auth/password/forgot	-	PasswordForgotIn	200 MessageOut	429
POST /auth/password/reset	-	PasswordResetIn	200 MessageOut	400, 422
GET /me	yes	-	200 UserOut	401
PATCH /me	yes	UserPatchIn	200 UserOut	422
POST /me/avatar	yes	multipart file	200 UserOut	422
POST /me/password	yes	PasswordChangeIn	200 MessageOut	400, 422
POST /me/terms	yes	TermsAcceptIn	201 MessageOut	
GET /me/addresses	yes	-	200 [AddressOut]	
POST /me/addresses	yes	AddressIn	201 AddressOut	422
GET /me/addresses/{id}	yes	-	200 AddressOut	404
PATCH /me/addresses/{id}	yes	AddressPatchIn	200 AddressOut	404, 422
POST /me/addresses/{id}/default	yes	-	200 AddressOut	404
DELETE /me/addresses/{id}	yes	-	204	404
GET /me/payment-methods	yes	-	200 [PaymentMethodOut]	
POST /me/payment-methods	yes	PaymentMethodIn	201 PaymentMethodOut	422
POST /me/payment-methods/{id}/default	yes	-	200 PaymentMethodOut	404
DELETE /me/payment-methods/{id}	yes	-	204	404
GET /me/wishlist	yes	-	200 [WishlistItemOut]	
PUT /me/wishlist/{productId}	yes	-	201 WishlistItemOut	404
DELETE /me/wishlist/{productId}	yes	-	204	
GET /me/orders?tab=	yes	-	200 Page[OrderSummaryOut]	
GET /me/orders/{id}	yes	-	200 OrderDetailOut	404
POST /me/orders/{id}/cancel	yes	CancelIn?	200 OrderDetailOut	404, 409
POST /me/orders/{id}/reorder	yes	-	200 CartOut	404
GET /health	-	-	200 {status: ok}	

Seller portal (Bearer token of a seller account)

Method + path	Auth	Request body	Success	Errors
GET /seller/application	seller	-	200 ApplicationOut	404
PATCH /seller/application/steps/basic-info	seller	BasicInfoIn	200 ApplicationOut	409, 422
PATCH /seller/application/steps/business-info	seller	BusinessInfoIn	200 ApplicationOut	409, 422
PATCH /seller/application/steps/representative	seller	RepresentativeIn	200 ApplicationOut	409, 422
PATCH /seller/application/steps/banking	seller	BankingIn	200 ApplicationOut	409, 422
POST /seller/application/directors	seller	DirectorIn	201 DirectorOut	409, 422
PATCH|DELETE /seller/application/directors/{id}	seller	DirectorIn	200 DirectorOut | 204	404, 409
GET /seller/application/documents	seller	-	200 [DocumentOut]	
POST /seller/application/documents	seller	multipart type + file	201 DocumentOut	409, 422
POST /seller/application/submit	seller	SubmitIn	200 ApplicationStatusOut	409, 422
POST /seller/application/resubmit	seller	ResubmitIn	200 ApplicationStatusOut	409, 422
GET /seller/application/status	seller	-	200 ApplicationStatusOut	404
GET /seller/store/slug-available?slug=	seller	-	200 SlugAvailableOut	
POST /seller/store/activate	seller	StoreActivateIn	200 StoreOut	409, 422
GET /seller/store	seller	-	200 StoreOut	404
GET /lookups/seller	-	-	200 SellerLookupsOut	
GET /lookups/banks?country=	-	-	200 [BankOut]	

Catalogue (public; send a Bearer token to get is_wishlisted)

Method + path	Auth	Request body	Success	Errors
GET /categories	-	-	200 [CategoryOut] (tree)	
GET /brands	-	-	200 [BrandOut]	
GET /products/filters	-	-	200 ProductFiltersOut	
GET /products?...	optional	query: ProductQuery	200 Page[ProductCardOut]	404 (category), 422
GET /products/featured | best-sellers | new-arrivals | top-discounted	optional	?limit=	200 [ProductCardOut]	
GET /products/{slug}	optional	-	200 ProductDetailOut	404
GET /products/{slug}/related	optional	?limit=	200 [ProductCardOut]	404
GET /products/{slug}/similar	optional	?limit=	200 [ProductCardOut]	404
GET /products/{slug}/frequently-bought-together	optional	?limit=	200 [ProductCardOut]	404
GET /search/suggest	-	?q=&limit;=	200 SuggestOut	422
GET /recommendations	optional	?limit= (+X-Cart-Token)	200 RecommendationsOut	
GET /seller/dashboard	store	?period=	200 DashboardOut	403
GET /seller/search	store	?q=&limit;=	200 PortalSearchOut	403
GET /seller/products	store	SellerProductQuery + page	200 page[SellerProductRowOut]	403
GET /seller/products/kpis	store	?period=	200 SellerProductKpisOut	
POST /seller/products	store	ProductCreateIn	201 SellerProductOut	422
GET|PATCH|DELETE /seller/products/{id}	store	ProductPatchIn	200 SellerProductOut / 204	404, 409, 422
PUT /seller/products/{id}/options	store	OptionsIn	200 SellerProductOut	404, 409, 422
PATCH /seller/products/{id}/variants/{vid}	store	VariantPatchIn	200 SellerVariantOut	404, 409, 422
POST /seller/products/{id}/images	store	multipart file, alt	201 SellerImageOut	404, 422
PUT /seller/products/{id}/images/order	store	ImageOrderIn	200 SellerProductOut	422
DELETE /seller/products/{id}/images/{iid}	store	-	204	404
PUT /seller/products/{id}/content	store	ContentIn	200 SellerProductOut	
POST /seller/products/{id}/status	store	StatusChangeIn	200 SellerProductOut	409, 422
POST /seller/products/{id}/duplicate	store	-	201 SellerProductOut	404
GET /seller/products/{id}/analytics	store	?period=	200 ProductAnalyticsOut	404
GET /admin/products	staff	?status=&page;	200 page[AdminProductRowOut]	
POST /admin/products/{id}/approve|reject	staff	RejectIn (reject)	200 MessageOut	404, 409
GET /seller/inventory	store	InventoryQuery + page	200 page[InventoryRowOut]	
GET /seller/inventory/kpis	store	-	200 InventoryKpisOut	
POST /seller/inventory/{vid}/adjust	store	StockAdjustIn	200 InventoryVariantOut	404, 409, 422
GET /seller/inventory/{vid}/movements	store	?limit=	200 [StockMovementOut]	404
GET /seller/inventory/export.csv	store	-	200 text/csv	
GET /seller/orders/overview	store	?period=	200 SellerOrdersOverviewOut	

Method + path	Auth	Request body	Success	Errors
GET /seller/orders	store	SellerOrderQuery + page	200 page[SellerOrderRowOut]	
GET /seller/orders/tabs	store	-	200 OrderTabsOut	
GET /seller/orders/{id}	store	-	200 SellerOrderDetailOut	404
POST /seller/orders/{id}/status	store	OrderStatusIn	200 SellerOrderDetailOut	404, 409
POST /seller/orders/{id}/cancel	store	OrderCancelIn	200 SellerOrderDetailOut	404, 409
GET /seller/orders/{id}/invoice	store	-	200 SellerInvoiceOut	404
GET /seller/orders/export.csv	store	-	200 text/csv	
GET /seller/customers/overview	store	?period=	200 CustomersOverviewOut	
GET /seller/customers	store	CustomerQuery + page	200 page[CustomerRowOut]	
GET /seller/customers/tabs	store	-	200 CustomerTabsOut	
GET /seller/customers/{key}	store	-	200 CustomerDetailOut	404
POST /seller/customers/{key}/notes	store	CustomerNoteIn	201 CustomerNoteOut	404
DELETE /seller/customers/{key}/notes/{id}	store	-	204	404
GET /seller/customers/export.csv	store	-	200 text/csv	
GET /products/{slug}/reviews?sort=	-	-	200 Page[ReviewOut]	404
GET /products/{slug}/reviews/summary	-	-	200 RatingBreakdownOut	404
POST /products/{slug}/reviews	optional	multipart ReviewIn + files[]	201 ReviewCreatedOut	404, 409, 422

Cart, checkout, tracking, support (guests: X-Cart-Token header)

Method + path	Auth	Request body	Success	Errors
GET /cart	optional	-	200 CartOut	
POST /cart/items	optional	CartItemIn	200 CartOut	404, 409
PATCH|DELETE /cart/items/{id}	optional	CartItemPatchIn	200 CartOut	404, 409
PATCH /cart	optional	CartPatchIn	200 CartOut	422
POST|DELETE /cart/voucher	optional	VoucherApplyIn	200 CartOut	404, 422
GET /delivery-methods	-	-	200 [DeliveryMethodOut]	
POST /checkout	optional	CheckoutIn	201 OrderPlacedOut	404, 409, 422
POST /orders/track	-	TrackIn	200 TrackOut	404
GET /banners?placement=	-	-	200 [BannerOut]	422
POST /newsletter/subscribe	-	NewsletterSubscribeIn	201 | 200 MessageOut	422
POST /contact	optional	ContactIn	201 ContactCreatedOut	422
GET /site-info	-	-	200 SiteInfoOut	

Admin (Bearer token of a staff account)

Method + path	Auth	Request body	Success	Errors
GET /admin/seller-applications?status=&account;_type=&q;=	staff	-	200 Page[AdminApplicationListOut]	
GET /admin/seller-applications/{id}	staff	-	200 AdminApplicationOut	404
POST .../{id}/start-review | approve | reject | request-info	staff	ApproveIn | RejectIn | RequestInfoIn	200 AdminApplicationOut	404, 409, 422
POST .../{id}/documents/{docId}/review	staff	DocumentReviewIn	200 DocumentOut	404
GET /admin/orders?status=&q;=	staff	-	200 Page[OrderSummaryOut]	
GET /admin/orders/{id}	staff	-	200 OrderDetailOut	404
POST /admin/orders/{id}/status	staff	AdminOrderStatusIn	200 OrderDetailOut	404, 409

Key schemas

Abridged - Swagger has every field. `Page[T]` means {items: T[], total, page, per_page, pages}.

```
UserOut {
  id: uuid, email, phone, full_name, first_name, last_name,
  gender: "male"|"female"|"other"|"prefer_not_to_say"|"",
  date_of_birth: "YYYY-MM-DD"|null, profession, country: "PK", city, address_line,
  status: "active"|"suspended"|"deleted", is_seller, is_staff,
  email_verified: bool, phone_verified: bool, member_since: ISO datetime, avatar_url|null
}
TokenOut  { access_token, refresh_token, token_type: "bearer", expires_in: 900 }
AuthOut   { user: UserOut, tokens: TokenOut }
AddressOut { id, type: "home"|"office"|"other", label, full_name, phone, street_address, unit_floor,
             city, province, postal_code, country, delivery_instructions, is_default, created_at, updated_at }
PaymentMethodOut { id, type: "card"|"paypal"|"apple_pay"|"google_pay"|"bank_transfer",
                   card_brand: "visa"|"mastercard"|"other"|"", card_holder_name, last4,
                   exp_month, exp_year, provider, is_default, display_name, created_at }
ProductCardOut { id, slug, name, category {id, name, slug}, brand, price_cents, compare_at_price_cents,
                 discount_percent, currency, image_url, stock_status, rating_avg, rating_count, is_wishlisted? }
ProductDetailOut = ProductCardOut + { short_description, description_title, description, images[],
                 option_types[{id, name, values[{id, value, swatch_hex}]}],
                 variants[{id, sku, option_value_ids[], price_cents, compare_at_price_cents, stock_qty, in_stock}],
                 key_benefits[{title, body}], usage_steps[{title, body}], store {id, display_name, slug},
                 ratings {average, count, distribution {5..1}} }
CartOut { id, cart_token?, items[{id, variant_id, product_slug, product_name, variant_summary, image_url,
          unit_price_cents, compare_at_price_cents, qty, line_total_cents, available_qty, in_stock}],
          order_note, voucher {code, type, value, label}?, delivery_method {id, name, eta_label, price_cents}?,
          summary {subtotal_cents, shipping_cents, discount_cents, total_cents, currency, item_count},
          can_checkout, problems[] }
OrderSummaryOut { id, order_number, status, status_label, group: ongoing|completed|cancelled, placed_at,
          total_cents, currency, payment_method_type, payment_status, items[...OrderItemOut], can_cancel, can_reor
ApplicationOut { id, application_number, account_type, status, status_headline, editable, current_step,
          steps[{step, name, required, complete, missing_fields[], missing_documents[]}], can_submit,
          basic_info, business_info, representative, banking (masked), directors[], documents[],
          open_info_request {message, items[{kind, target, instruction, resolved}]}?, store? }
```

8. Data conventions for the frontend

Topic	Convention
IDs	UUID strings everywhere (users, addresses, products, orders...)
Dates/times	ISO 8601 in UTC, e.g. 2026-09-12T20:39:00Z. Dates without time are YYYY-MM-DD. Format for display client-side.
Money	Integers in minor units: price_cents = 2599 means $25.99, alongside a currency code. Never send floats.
Phone numbers	E.164 (+923001234567). The API accepts spaces/dashes/parentheses and normalises; display formatting is the client's job.
Email	Lower-cased by the server
Enums	Lower-snake_case strings exactly as listed in Swagger. Unknown values are rejected with 422.
Optional text fields	Come back as "" (empty string) when unset, not null; nullable dates/numbers come back as null.
Partial updates	Use PATCH and send only the keys that changed. Sending null clears a field.
Lists	Small lists (addresses, cards, wishlist) are plain arrays. Products, reviews, orders and admin queues are paginated: {items, total, page, per_page, pages}; per_page defaults to 12 (max 100).
Guest cart	Keep the cart_token from the first /cart response and send it as X-Cart-Token; send it once together with the Bearer token after login to merge.
Masked PII	Sellers see their own ID / account numbers masked (id_passport_number_masked, account_number_last4); only staff endpoints return the full values.
File uploads	multipart/form-data with the field named 'file' (avatar) or per the endpoint; size/type limits are enforced server-side and returned as 422.
Idempotency	Setting a default twice, or verifying an already-verified email, is safe.

9. Working with the API day to day

Trying calls in Swagger

- Open /api/v1/docs, expand **POST /auth/register**, 'Try it out', submit. Copy `tokens.access_token` from the response.
- Click **Authorize** (top right), paste the access token (without 'Bearer '), then any /me endpoint works.
- For OTP codes, look at the terminal running `runserver`.

Generating a typed client

```
# TypeScript types straight from the live spec
npx openapi-typescript http://127.0.0.1:8000/api/v1/openapi.json -o src/api/schema.d.ts
```

Regenerate the client whenever the backend announces a schema change; the spec is the contract.

Where to look when something is unclear

Question	Source
What fields does endpoint X accept and return?	Swagger /api/v1/docs (authoritative), then backend/asknget/<app>/schemas.py
Why did the API reject my payload?	The 422 body's errors[]; rules are in schemas.py, business rules in services.py
Which tables/fields exist and why?	docs/data-model.md (every table annotated with the Figma label it came from)
Has the design changed since the backend was built?	python docs/figma/snapshot.py --check, then git diff docs/figma/
Open product questions (currency, guest checkout, multi-vendor shipping...)	docs/data-model.md section 10

Roadmap (subject to sprint planning)

Order	Scope	Depends on
1	Payment provider: client-side tokenisation, POST /checkout/{id}/pay, webhooks -> payment_status/paid, saved cards, refunds on cancel	Provider decision (see data-model.md section 10)
2	Seller portal remainder: Returns flow, Chat, Analytics page, Notifications, Team/Managers (sub-roles), invoice PDF, social login	Design for those screens + answers to Q20-Q34
3	Review moderation UI, category/brand management endpoints (Django admin covers these today)	-
4	Vendor sub-roles/permissions, premium tier, vendor-customer messaging	Product decisions + design
5	Email/SMS providers; more worker jobs (category counts, rating refresh, cart expiry)	Worker service exists; add commands
6	LLM query understanding ('gift under $50 for dry skin' -> filters) and a streaming shopping assistant	Claude API key; ASGI (uvicorn) for streaming - only those endpoints go async

Repository: https://github.com/AunSyedShah/asknget | Backend README: backend/README.md | This document is generated by docs/build_developer_guide.py - edit the script, not the PDF.